

Credit Card Fraud Detection

SQL Analytics Project

Schema design • ETL • 57 documented queries
MySQL 8.0 / MySQL Workbench

Dataset: Kaggle — Credit Card Transactions Fraud Detection (~1.3M transactions)
Author: Alkendra

Project Overview

This project takes a flat, denormalized credit-card transactions file and turns it into a clean, queryable analytics database. It is organised in two parts: first the data model (schema design, loading, and normalization), then a library of 57 analytical queries. Every query is documented in a fixed format — a **Question** stating why the query exists, the **query** itself, and a **What we get** box describing the insight the result delivers.

Dataset

Source: Kaggle kartik2112/fraud-detection (files `fraudTrain.csv` + `fraudTest.csv`, combined for the full ~1.3 million rows). Each row is one transaction; the target column `is_fraud` is 1 for fraud and 0 otherwise. Fraud is a highly imbalanced class at roughly 0.5% of all transactions.

Normalized Schema

The wide source file is modelled into three related tables. Customers and Merchants are dimension-style tables; Transactions is the fact table linking them.

Table	Primary Key	Foreign Keys
Customers	cc_num	—
Merchants	merchant_id (surrogate, AUTO_INCREMENT)	
Transactions	trans_num	cc_num → Customers, merchant_id → Merchants

Relationship: Customers (1) —< Transactions >— (1) Merchants. One customer has many transactions; one merchant has many transactions.

How to Run

1. Download the dataset from Kaggle and note the CSV file paths.
2. Run **01_schema.sql** — creates the database, the staging table, and the three normalized tables with their keys.
3. Run **02_load_data.sql** — loads the CSV(s) and populates the tables. Read the `secure_file_priv` notes first if LOAD DATA errors.
4. Run the query files in order (03 through 09). Each file is self-contained.

Topic Coverage

Area	Topics
Foundations	SELECT, WHERE, GROUP BY, HAVING, ORDER BY, aggregates
Joins	INNER, LEFT, RIGHT, SELF join
Functions	String, Date/Time, NULL handling (COALESCE, IFNULL, NULLIF)
Logic / reshape	CASE WHEN, PIVOT (conditional aggregation)
Subqueries	Nested, correlated, derived tables

Window functions	ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, FIRST/LAST_VALUE, running totals, moving
CTEs	Simple, chained, recursive
DB objects	Views, stored procedures (IN/OUT), transactions (COMMIT/ROLLBACK)
Performance	EXPLAIN, single & composite indexes
Applied analytics	Haversine distance, z-score anomaly, cohort analysis, velocity checks

Part 1 — Schema Design (01_schema.sql)

The source CSV is a single wide table. We first load it as-is into a **staging table**, then split it into the three normalized tables. Keeping the raw import separate from the modelled data is standard ETL practice. Below is the full schema script.

```
-- =====
-- PROJECT : Credit Card Fraud Detection - SQL Analytics
-- FILE    : 01_schema.sql
-- PURPOSE : Create the database and a normalized 3-table schema
--           (Customers, Merchants, Transactions) from the flat
--           Kaggle "Credit Card Transactions Fraud Detection" dataset.
-- ENGINE  : MySQL 8.0+ (tested on MySQL Workbench)
-- AUTHOR  : Alkendra
-- =====

-- -----
-- STEP 1 : Create and select the database
-- -----
DROP DATABASE IF EXISTS fraud_analytics;
CREATE DATABASE fraud_analytics;
USE fraud_analytics;

-- -----
-- STEP 2 : STAGING TABLE
-- WHY    : The Kaggle CSV is a single wide (denormalized) file. We first
--          load it AS-IS into a staging table, then split it into three
--          clean, normalized tables. This is standard ETL practice and
--          keeps the raw import separate from the modelled data.
-- -----
DROP TABLE IF EXISTS stg_transactions;
CREATE TABLE stg_transactions (
    row_id          INT,                -- the unnamed index column in the CSV
    trans_date_trans_time DATETIME,
    cc_num          BIGINT,
    merchant        VARCHAR(120),      -- note: raw values are prefixed with "fraud_"
    category        VARCHAR(60),
    amt            DECIMAL(10,2),
    first          VARCHAR(60),
    last           VARCHAR(60),
    gender          CHAR(1),
    street          VARCHAR(120),
    city            VARCHAR(60),
    state           VARCHAR(5),
    zip             VARCHAR(10),
    lat             DECIMAL(9,6),      -- customer home latitude
    `long`          DECIMAL(9,6),      -- customer home longitude (backticked: LONG is reserved-i
    city_pop        INT,
    job             VARCHAR(120),
    dob             DATE,
    trans_num       VARCHAR(64),
    unix_time       BIGINT,
    merch_lat       DECIMAL(9,6),      -- merchant latitude AT transaction time (varies per txn)
    merch_long      DECIMAL(9,6),      -- merchant longitude at transaction time
    is_fraud        TINYINT(1)
);
```

```

-- -----
-- STEP 3 : NORMALIZED TABLES
-- -----

-- 3a. CUSTOMERS
-- WHY : One row per credit-card holder. In this dataset cc_num is unique
--       per customer, so it is a natural PRIMARY KEY.
DROP TABLE IF EXISTS Customers;
CREATE TABLE Customers (
    cc_num      BIGINT      PRIMARY KEY,
    first       VARCHAR(60),
    last        VARCHAR(60),
    gender      CHAR(1),
    street      VARCHAR(120),
    city        VARCHAR(60),
    state       VARCHAR(5),
    zip         VARCHAR(10),
    lat         DECIMAL(9,6),
    `long`      DECIMAL(9,6),
    city_pop    INT,
    job         VARCHAR(120),
    dob         DATE
);

-- 3b. MERCHANTS
-- WHY : The raw file has no merchant ID, only a repeated merchant NAME.
--       We create a surrogate AUTO_INCREMENT key (merchant_id) as the
--       PRIMARY KEY. Each merchant maps to exactly one category, so
--       category lives here. (merch_lat/merch_long stay in Transactions
--       because they change per transaction - they are GPS-at-purchase.)
DROP TABLE IF EXISTS Merchants;
CREATE TABLE Merchants (
    merchant_id INT AUTO_INCREMENT PRIMARY KEY,
    merchant_name VARCHAR(120),
    category     VARCHAR(60),
    UNIQUE KEY uq_merchant_name (merchant_name) -- prevents duplicate merchants
);

-- 3c. TRANSACTIONS
-- WHY : The fact table - one row per transaction. trans_num is already
--       unique in the dataset, so it is the PRIMARY KEY. Two FOREIGN KEYS
--       link each transaction to its customer and its merchant.
DROP TABLE IF EXISTS Transactions;

CREATE TABLE Transactions (
    trans_num      VARCHAR(64) PRIMARY KEY,
    cc_num         BIGINT,
    merchant_id    INT,
    trans_date_trans_time DATETIME,
    unix_time      BIGINT,
    amt            DECIMAL(10,2),
    merch_lat      DECIMAL(9,6),
    merch_long     DECIMAL(9,6),
    is_fraud       TINYINT(1),
    CONSTRAINT fk_txn_customer
        FOREIGN KEY (cc_num) REFERENCES Customers(cc_num),
    CONSTRAINT fk_txn_merchant
        FOREIGN KEY (merchant_id) REFERENCES Merchants(merchant_id)
);

-- -----
-- Schema is ready. Next: run 02_load_data.sql to import the CSV and
-- populate these three tables from the staging table.
-- -----

```

Part 2 — Loading & Normalization (02_load_data.sql)

This step imports the CSV into staging and then populates the normalized tables. The most common blocker here is MySQL's `secure_file_priv` restriction, which controls where `LOAD DATA INFILE` may read files from. The script documents both fixes: placing the file in the permitted server folder, or using `LOAD DATA LOCAL INFILE` (recommended in Workbench) which reads the file from your own machine. During normalization the script also strips the `fraud_` prefix that the dataset attaches to every merchant name (a data quirk, not a fraud flag).

```
-- =====
-- FILE      : 02_load_data.sql
-- PURPOSE   : Load the dataset into the staging table, then populate the
--             three normalized tables. Includes a full workaround for the
--             classic MySQL "secure_file_priv" error.
-- DATASET   : data/fraud_data.csv (included with this project - 100k rows).
--             Its columns match the Kaggle kartik2112/fraud-detection layout
--             exactly, so if you prefer the real ~1.3M-row data you can load
--             fraudTrain.csv + fraudTest.csv with the SAME statements below.
-- =====

USE fraud_analytics;

-- -----
-- IMPORTANT - THE secure_file_priv PROBLEM (read before loading)
-- -----
-- MySQL restricts where LOAD DATA INFILE can read files from. Check yours:
--
--     SHOW VARIABLES LIKE 'secure_file_priv';
--
-- You will get one of three results:
-- (a) A folder path -> you can ONLY load files placed inside that folder.
-- (b) Empty string  -> you can load from anywhere.
-- (c) NULL          -> file import via LOAD DATA INFILE is DISABLED.
--
-- TWO WAYS TO SOLVE IT:
--
-- OPTION 1 (server-side, needs the folder):
-- Copy fraud_data.csv into the folder shown by secure_file_priv, then
-- use LOAD DATA INFILE (no LOCAL keyword). See BLOCK A below.
--
-- OPTION 2 (client-side, RECOMMENDED for Workbench):
-- Use LOAD DATA LOCAL INFILE - reads the file from YOUR machine, so the
-- secure_file_priv folder does not matter. It needs local_infile enabled
-- on BOTH server and client:
-- 1. On server: SET GLOBAL local_infile = 1;
-- 2. In Workbench: Edit connection > Connection tab > Advanced >
--    add OPT_LOCAL_INFILE=1 then reconnect.
-- See BLOCK B below.
-- -----

SET GLOBAL local_infile = 1;  -- enable client-side loading (Option 2)
```

```

-- =====
-- BLOCK A : LOAD DATA INFILE (server-side - use ONLY with Option 1)
-- Replace the path with your secure_file_priv folder + filename.
-- =====
/*
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/fraud_data.csv'
INTO TABLE stg_transactions
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(row_id, @trans_dt, cc_num, merchant, category, amt, first, last, gender,
 street, city, state, zip, lat, `long`, city_pop, job, @dob_str,
 trans_num, unix_time, merch_lat, merch_long, is_fraud)
SET trans_date_trans_time = STR_TO_DATE(@trans_dt, '%Y-%m-%d %H:%i:%s'),
    dob = STR_TO_DATE(@dob_str, '%Y-%m-%d');
*/

-- =====
-- BLOCK B : LOAD DATA LOCAL INFILE (client-side - RECOMMENDED)
-- Point the path at wherever fraud_data.csv sits on your own laptop.
-- On Windows use forward slashes, e.g. 'C:/Users/Alkendra/Downloads/fraud_data.csv'
-- =====
LOAD DATA LOCAL INFILE 'C:/Users/Alkendra/Downloads/fraud_data.csv'
INTO TABLE stg_transactions
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(row_id, @trans_dt, cc_num, merchant, category, amt, first, last, gender,
 street, city, state, zip, lat, `long`, city_pop, job, @dob_str,
 trans_num, unix_time, merch_lat, merch_long, is_fraud)
SET trans_date_trans_time = STR_TO_DATE(@trans_dt, '%Y-%m-%d %H:%i:%s'),
    dob = STR_TO_DATE(@dob_str, '%Y-%m-%d');

-- (Using the real Kaggle data instead? Run the same block once per file -
-- once for fraudTrain.csv and once for fraudTest.csv - to reach ~1.3M rows.)

-- Sanity check the staging load:
SELECT COUNT(*) AS staged_rows FROM stg_transactions;

-- =====
-- STEP 4 : POPULATE THE NORMALIZED TABLES FROM STAGING
-- =====

```

```

-- 4a. CUSTOMERS
-- WHY : One row per unique cc_num. GROUP BY collapses the many
--        transaction rows of each customer down to a single profile row.
INSERT INTO Customers (cc_num, first, last, gender, street, city, state,
                        zip, lat, `long`, city_pop, job, dob)
SELECT cc_num, first, last, gender, street, city, state,
       zip, lat, `long`, city_pop, job, dob
FROM stg_transactions
GROUP BY cc_num, first, last, gender, street, city, state,
       zip, lat, `long`, city_pop, job, dob;

-- 4b. MERCHANTS
-- WHY : One row per unique merchant name. We strip the "fraud_" prefix
--        that the dataset attaches to every merchant name (it is a data
--        quirk, NOT an indicator of fraud) using REPLACE.
INSERT INTO Merchants (merchant_name, category)
SELECT DISTINCT
    REPLACE(merchant, 'fraud_', '') AS merchant_name,
    category
FROM stg_transactions;

-- 4c. TRANSACTIONS
-- WHY : The fact rows. We JOIN staging to Merchants (on the cleaned name)
--        to translate the merchant name into its surrogate merchant_id.
INSERT INTO Transactions (trans_num, cc_num, merchant_id,
                          trans_date_trans_time, unix_time, amt,
                          merch_lat, merch_long, is_fraud)
SELECT s.trans_num,
       s.cc_num,
       m.merchant_id,
       s.trans_date_trans_time,
       s.unix_time,
       s.amt,
       s.merch_lat,
       s.merch_long,
       s.is_fraud
FROM stg_transactions s
JOIN Merchants m
    ON REPLACE(s.merchant, 'fraud_', '') = m.merchant_name;

-- -----
-- STEP 5 : VERIFY ROW COUNTS
-- WHAT WE EXPECT (with the included file):
--   Customers = 500 | Merchants = 200 | Transactions ~ 100,000
-- -----
SELECT 'Customers'    AS table_name, COUNT(*) AS rows_loaded FROM Customers
UNION ALL
SELECT 'Merchants',   COUNT(*) FROM Merchants
UNION ALL
SELECT 'Transactions', COUNT(*) FROM Transactions;

```

```

-- Optional: once you have confirmed the counts, you can drop staging.
-- DROP TABLE stg_transactions;

```

Part 3 — The Query Library (57 Queries)

Each query below is presented in three parts: the **Question** (why we run it), the **SQL**, and a green **What we get** box (the insight). Queries are grouped by topic and numbered Q1–Q57.

Foundational Queries: SELECT, Filtering, Aggregates & Joins

Q1

Question: What is the overall scale of fraud in the portfolio? (total transactions, total frauds, and the fraud rate %)

```
SELECT
  COUNT(*)                AS total_txns,
  SUM(is_fraud)           AS total_frauds,
  ROUND(100 * SUM(is_fraud) / COUNT(*), 4) AS fraud_rate_pct
FROM Transactions;
```

WHAT WE GET

Establishes the baseline. Fraud is a highly imbalanced class (~0.5-0.6% of all transactions) - every later % is judged against this baseline rate.

Q2

Question: Which spending categories carry the most fraud, by count and rate?

```
SELECT
  m.category,
  COUNT(*)                AS total_txns,
  SUM(t.is_fraud)         AS fraud_txns,
  ROUND(100 * SUM(t.is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
GROUP BY m.category
ORDER BY fraud_rate_pct DESC;
```

WHAT WE GET

Ranks categories by risk. Online-style categories such as shopping_net and grocery_pos usually top the fraud-rate list, which is where monitoring effort should focus first.

Q3

Question: Which categories are WORSE than the portfolio's average fraud rate? (demonstrates HAVING to filter on an aggregate)

```

SELECT
    m.category,
    ROUND(100 * SUM(t.is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
GROUP BY m.category
HAVING fraud_rate_pct > (
    SELECT 100 * SUM(is_fraud) / COUNT(*) FROM Transactions
)
ORDER BY fraud_rate_pct DESC;

```

WHAT WE GET

Isolates the "above-average risk" categories - a shortlist for tighter rules, without hard-coding any threshold number.

Q4

Question: What do the 10 largest transactions look like? (WHERE-free ranking with ORDER BY + LIMIT)

```

SELECT trans_num, cc_num, amt, is_fraud, trans_date_trans_time
FROM Transactions
ORDER BY amt DESC
LIMIT 10;

```

WHAT WE GET

High-ticket transactions are disproportionately likely to be fraudulent; a quick eyeball of is_fraud here shows whether the biggest amounts skew fraudulent.

Q5

Question: Do fraudulent transactions have a different average amount than legitimate ones?

```

SELECT
    CASE WHEN is_fraud = 1 THEN 'Fraud' ELSE 'Legitimate' END AS txn_type,
    COUNT(*) AS txns,
    ROUND(AVG(amt), 2) AS avg_amount,
    ROUND(MAX(amt), 2) AS max_amount,
    ROUND(MIN(amt), 2) AS min_amount
FROM Transactions
GROUP BY is_fraud;

```

WHAT WE GET

Fraudulent transactions almost always show a much HIGHER average amount - amount alone is a strong predictive signal.

Q6

Question: Is fraud distributed differently across genders?

```

SELECT
    c.gender,
    COUNT(*) AS total_txns,
    SUM(t.is_fraud) AS fraud_txns,
    ROUND(100 * SUM(t.is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions t
JOIN Customers c ON t.cc_num = c.cc_num
GROUP BY c.gender;

```

WHAT WE GET

Checks whether gender is associated with fraud exposure. Usually the rates are close, telling us gender is a weak predictor on its own.

Q7

Question: (INNER JOIN) List fraudulent transactions with the customer's full name and the merchant they hit.

```
SELECT
    t.trans_num,
    CONCAT(c.first, ' ', c.last) AS customer_name,
    m.merchant_name,
    m.category,
    t.amt,
    t.trans_date_trans_time
FROM Transactions t
INNER JOIN Customers c ON t.cc_num = c.cc_num
INNER JOIN Merchants m ON t.merchant_id = m.merchant_id
WHERE t.is_fraud = 1
ORDER BY t.amt DESC
LIMIT 20;
```

WHAT WE GET

The core investigative view - turns opaque IDs into a human-readable fraud case list an analyst can act on.

Q8

Question: (LEFT JOIN) Do we have any merchants that never recorded a single transaction? (data-quality / coverage check)

```
SELECT
    m.merchant_id,
    m.merchant_name,
    COUNT(t.trans_num) AS txn_count
FROM Merchants m
LEFT JOIN Transactions t ON m.merchant_id = t.merchant_id
GROUP BY m.merchant_id, m.merchant_name
HAVING txn_count = 0;
```

WHAT WE GET

A LEFT JOIN keeps every merchant even with zero matches. An empty result set here confirms every merchant is active (good data integrity); any rows would flag orphan merchants.

Q9

Question: (RIGHT JOIN) Same coverage idea, expressed from the transaction side - confirm every transaction maps to a known merchant.

```
SELECT COUNT(*) AS txns_without_merchant
FROM Merchants m
RIGHT JOIN Transactions t ON m.merchant_id = t.merchant_id
WHERE m.merchant_id IS NULL;
```

WHAT WE GET

A RIGHT JOIN keeps every transaction. A count of 0 proves referential integrity - no transaction points at a missing merchant. (RIGHT JOIN is just a LEFT JOIN read backwards.)

Q10

Question: Which customer JOBS have the highest fraud exposure?

```
SELECT
  c.job,
  COUNT(*) AS total_txns,
  SUM(t.is_fraud) AS fraud_txns,
  ROUND(100 * SUM(t.is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions t
JOIN Customers c ON t.cc_num = c.cc_num
GROUP BY c.job
HAVING total_txns > 500 -- ignore tiny, noisy job groups
ORDER BY fraud_rate_pct DESC
LIMIT 15;
```

WHAT WE GET

Surfaces occupation segments with elevated fraud - useful demographic context, filtered to statistically meaningful group sizes with HAVING.

Q11

Question: Which STATES see the most fraud?

```
SELECT
  c.state,
  COUNT(*) AS total_txns,
  SUM(t.is_fraud) AS fraud_txns,
  ROUND(100 * SUM(t.is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions t
JOIN Customers c ON t.cc_num = c.cc_num
GROUP BY c.state
ORDER BY fraud_txns DESC
LIMIT 10;
```

WHAT WE GET

Geographic concentration of fraud - drives where regional risk teams or state-level rules should be prioritised.

Q12

Question: (SELF JOIN) Find pairs of customers who live in the same city and state - a peer group used later for behavioural comparison.

```
SELECT
  a.city,
  a.state,
  CONCAT(a.first, ' ', a.last) AS customer_a,
  CONCAT(b.first, ' ', b.last) AS customer_b
FROM Customers a
JOIN Customers b
  ON a.city = b.city
  AND a.state = b.state
  AND a.cc_num < b.cc_num -- avoid self-pairing & mirror duplicates
ORDER BY a.state, a.city
LIMIT 20;
```

WHAT WE GET

A SELF JOIN relates a table to itself. The `a.cc_num < b.cc_num` trick returns each pair once. This defines "local peers" for spotting a customer whose behaviour deviates from their area.

String Functions, Date/Time Functions & NULL Handling

String Functions

Q13

Question: Build a clean customer directory: full name (proper case), initials, and the length of the full name.

```
SELECT
  cc_num,
  CONCAT(first, ' ', last) AS full_name,
  CONCAT(UPPER(LEFT(first,1)), UPPER(LEFT(last,1))) AS initials,
  CHAR_LENGTH(CONCAT(first, ' ', last)) AS name_length
FROM Customers
LIMIT 15;
```

WHAT WE GET

Demonstrates CONCAT, UPPER, LEFT, CHAR_LENGTH to reshape raw name fields into report-ready strings - the kind of tidy output a dashboard or export needs.

Q14

Question: Confirm the "fraud_" prefix was stripped correctly, and show the cleaned merchant name in upper case for a report header.

```
SELECT
  merchant_id,
  merchant_name,
  UPPER(merchant_name) AS display_name,
  SUBSTRING(merchant_name, 1, 10) AS short_name
FROM Merchants
LIMIT 15;
```

WHAT WE GET

Verifies the REPLACE cleaning done at load time and shows SUBSTRING for truncating long merchant names in tight UI space.

Date / Time Functions

Q15

Question: How old is each customer, and how does fraud rate vary by age band?

```

SELECT
    age_band,
    COUNT(*)                               AS total_txns,
    SUM(is_fraud)                          AS fraud_txns,
    ROUND(100 * SUM(is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM (
    SELECT
        t.is_fraud,
        CASE
            WHEN TIMESTAMPDIFF(YEAR, c.dob, t.trans_date_trans_time) < 30 THEN '1) Under 30'
            WHEN TIMESTAMPDIFF(YEAR, c.dob, t.trans_date_trans_time) < 45 THEN '2) 30-44'
            WHEN TIMESTAMPDIFF(YEAR, c.dob, t.trans_date_trans_time) < 60 THEN '3) 45-59'
            ELSE '4) 60+'
        END AS age_band
    FROM Transactions t
    JOIN Customers c ON t.cc_num = c.cc_num
) AS aged
GROUP BY age_band
ORDER BY age_band;

```

WHAT WE GET

TIMESTAMPDIFF computes age at transaction time. Reveals which age groups are targeted most - often older cardholders show a higher fraud rate.

Q16

Question: At what HOUR of day does fraud peak?

```

SELECT
    HOUR(trans_date_trans_time)           AS txn_hour,
    COUNT(*)                             AS total_txns,
    SUM(is_fraud)                         AS fraud_txns,
    ROUND(100 * SUM(is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions
GROUP BY HOUR(trans_date_trans_time)
ORDER BY fraud_rate_pct DESC;

```

WHAT WE GET

HOUR() exposes the time-of-day signature. Fraud famously spikes in the late-night / early-morning hours (roughly 22:00-03:00) when victims are asleep.

Q17

Question: Which DAY OF WEEK carries the most fraud?

```

SELECT
    DAYNAME(trans_date_trans_time)        AS weekday,
    COUNT(*)                             AS total_txns,
    SUM(is_fraud)                         AS fraud_txns,
    ROUND(100 * SUM(is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions
GROUP BY DAYNAME(trans_date_trans_time), DAYOFWEEK(trans_date_trans_time)
ORDER BY DAYOFWEEK(trans_date_trans_time);

```

WHAT WE GET

DAYNAME() gives a weekly pattern; DAYOFWEEK is used in ORDER BY so the days come out Sun->Sat rather than alphabetically.

Q18

Question: What is the monthly fraud trend across the dataset?

```
SELECT
    DATE_FORMAT(trans_date_trans_time, '%Y-%m') AS ym,
    COUNT(*) AS total_txns,
    SUM(is_fraud) AS fraud_txns,
    ROUND(100 * SUM(is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions
GROUP BY DATE_FORMAT(trans_date_trans_time, '%Y-%m')
ORDER BY ym;
```

WHAT WE GET

DATE_FORMAT buckets to month. This is the headline trend line - is fraud rising, falling, or seasonal over the period?

Null Handling

Q19

Question: (COALESCE) Show customers, replacing any missing job with a readable placeholder so reports never display blank cells.

```
SELECT
    cc_num,
    CONCAT(first, ' ', last) AS customer_name,
    COALESCE(NULLIF(TRIM(job), ''), 'Not Provided') AS job_clean
FROM Customers
LIMIT 20;
```

WHAT WE GET

COALESCE returns the first non-NULL value; wrapping NULLIF around TRIM(job) also converts empty strings " into NULL first, so both blanks AND nulls become 'Not Provided'.

Q20

Question: (NULLIF) Safely compute fraud rate per state WITHOUT risking a divide-by-zero error.

```
SELECT
    c.state,
    SUM(t.is_fraud) AS fraud_txns,
    COUNT(*) AS total_txns,
    ROUND(100 * SUM(t.is_fraud) / NULLIF(COUNT(*), 0), 3) AS fraud_rate_pct
FROM Transactions t
JOIN Customers c ON t.cc_num = c.cc_num
GROUP BY c.state
ORDER BY fraud_rate_pct DESC
LIMIT 10;
```

WHAT WE GET

NULLIF(COUNT(*),0) turns a zero denominator into NULL, so the division yields NULL instead of throwing an error - a defensive pattern for any computed ratio.

Q21

Question: (IFNULL) Produce a per-customer fraud summary where customers who have never been defrauded still show 0, not a blank.

```
SELECT
  c.cc_num,
  CONCAT(c.first, ' ', c.last) AS customer_name,
  IFNULL(SUM(t.is_fraud), 0) AS fraud_txns
FROM Customers c
LEFT JOIN Transactions t ON c.cc_num = t.cc_num
GROUP BY c.cc_num, customer_name
ORDER BY fraud_txns DESC
LIMIT 15;
```

WHAT WE GET

IFNULL(x,0) guarantees a numeric result even for customers with no matching (or no fraudulent) transactions - keeps the summary clean and chart-ready.

CASE WHEN, PIVOT & Subqueries

Case When

Q22

Question: Bucket every transaction into an amount tier and see how fraud concentrates across tiers.

```
SELECT
  CASE
    WHEN amt < 10 THEN '1) < $10'
    WHEN amt < 50 THEN '2) $10-49'
    WHEN amt < 200 THEN '3) $50-199'
    WHEN amt < 1000 THEN '4) $200-999'
    ELSE '5) $1000+'
  END AS amount_tier,
  COUNT(*) AS total_txns,
  SUM(is_fraud) AS fraud_txns,
  ROUND(100 * SUM(is_fraud) / COUNT(*), 3) AS fraud_rate_pct
FROM Transactions
GROUP BY amount_tier
ORDER BY amount_tier;
```

WHAT WE GET

CASE turns a continuous amount into labelled tiers. Fraud rate climbs sharply in the highest tier - big-ticket transactions are far riskier per transaction.

Q23

Question: Build a category-by-quarter fraud matrix: one row per category, one column per quarter. (This is what "PIVOT" means in MySQL.)

```

SELECT
    m.category,
    SUM(CASE WHEN QUARTER(t.trans_date_trans_time)=1 THEN t.is_fraud ELSE 0 END) AS q1_fraud,
    SUM(CASE WHEN QUARTER(t.trans_date_trans_time)=2 THEN t.is_fraud ELSE 0 END) AS q2_fraud,
    SUM(CASE WHEN QUARTER(t.trans_date_trans_time)=3 THEN t.is_fraud ELSE 0 END) AS q3_fraud,
    SUM(CASE WHEN QUARTER(t.trans_date_trans_time)=4 THEN t.is_fraud ELSE 0 END) AS q4_fraud
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
GROUP BY m.category
ORDER BY SUM(t.is_fraud) DESC;

```

WHAT WE GET

SUM(CASE...) rotates rows into columns - the classic MySQL PIVOT. Instantly shows whether any category's fraud is seasonal (spikes in one quarter).

Q24

Question: Pivot transaction counts: gender (rows) x amount tier (columns).

```

SELECT
    c.gender,
    SUM(CASE WHEN t.amt < 50 THEN 1 ELSE 0 END) AS tier_under_50,
    SUM(CASE WHEN t.amt >= 50 AND t.amt < 200 THEN 1 ELSE 0 END) AS tier_50_199,
    SUM(CASE WHEN t.amt >= 200 THEN 1 ELSE 0 END) AS tier_200_plus,
    COUNT(*) AS total
FROM Transactions t
JOIN Customers c ON t.cc_num = c.cc_num
GROUP BY c.gender;

```

WHAT WE GET

A two-dimensional summary in a single scan - compares male vs female spending distribution across amount tiers without three separate queries.

Subqueries

Q25

Question: (Nested / non-correlated) Which transactions are larger than the overall average transaction amount?

```

SELECT trans_num, cc_num, amt, is_fraud
FROM Transactions
WHERE amt > (SELECT AVG(amt) FROM Transactions)
ORDER BY amt DESC
LIMIT 20;

```

WHAT WE GET

The inner query runs once and returns a single number; the outer query filters against it. A simple "above average" gate.

Q26

Question: (Correlated) Which transactions exceed the individual customer's OWN average spend? (personalised anomaly, not a global threshold)

```

SELECT
    t.trans_num,
    t.cc_num,
    t.amt,
    t.is_fraud
FROM Transactions t
WHERE t.amt > (
    SELECT AVG(t2.amt)
    FROM Transactions t2
    WHERE t2.cc_num = t.cc_num      -- correlation: inner depends on outer row
)
AND t.is_fraud = 1
ORDER BY t.amt DESC
LIMIT 20;

```

WHAT WE GET

A correlated subquery re-evaluates per customer, so "large" is relative to that person's normal behaviour - a much stronger fraud signal than one flat threshold.

Q27

Question: (Correlated) Which merchants have a fraud rate above the average fraud rate of their OWN category?

```

SELECT
    m.merchant_name,
    m.category,
    ROUND(100 * SUM(t.is_fraud)/COUNT(*), 3) AS merchant_fraud_rate
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
GROUP BY m.merchant_id, m.merchant_name, m.category
HAVING merchant_fraud_rate > (
    SELECT 100 * SUM(t2.is_fraud)/COUNT(*)
    FROM Transactions t2
    JOIN Merchants m2 ON t2.merchant_id = m2.merchant_id
    WHERE m2.category = m.category      -- compare each merchant to its category
)
ORDER BY merchant_fraud_rate DESC
LIMIT 20;

```

WHAT WE GET

Flags merchants that are risk outliers *within their peer category* - fairer than comparing a gas station to an online store on one global number.

Q28

Question: (Subquery in FROM) Rank customers by total spend using a derived table, then keep only the top spenders.

```

SELECT customer_name, total_spend, txn_count
FROM (
    SELECT
        c.cc_num,
        CONCAT(c.first, ' ', c.last) AS customer_name,
        ROUND(SUM(t.amt), 2) AS total_spend,
        COUNT(*) AS txn_count
    FROM Transactions t
    JOIN Customers c ON t.cc_num = c.cc_num
    GROUP BY c.cc_num, customer_name
) AS spend_summary
WHERE total_spend > 50000
ORDER BY total_spend DESC
LIMIT 15;

```

WHAT WE GET

A derived table (subquery in FROM) lets us aggregate first, then filter/sort the aggregated result - something WHERE cannot do directly on an aggregate.

Window Functions

Q29

Question: (ROW_NUMBER) Get the single MOST RECENT transaction per customer.

```

SELECT *
FROM (
    SELECT
        t.cc_num,
        t.trans_num,
        t.amt,
        t.trans_date_trans_time,
        ROW_NUMBER() OVER (
            PARTITION BY t.cc_num
            ORDER BY t.trans_date_trans_time DESC
        ) AS rn
    FROM Transactions t
) ranked
WHERE rn = 1
LIMIT 20;

```

WHAT WE GET

ROW_NUMBER numbers rows within each customer partition; rn=1 picks each customer's latest activity - the classic "top-N-per-group" / most-recent-record pattern.

Q30

Question: (RANK vs DENSE_RANK) Rank merchants by total fraud count and see how the two ranking functions treat ties differently.

```

SELECT
    m.merchant_name,
    SUM(t.is_fraud)                                AS fraud_count,
    RANK() OVER (ORDER BY SUM(t.is_fraud) DESC)    AS rnk,
    DENSE_RANK() OVER (ORDER BY SUM(t.is_fraud) DESC) AS dense_rnk
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
GROUP BY m.merchant_id, m.merchant_name
ORDER BY fraud_count DESC
LIMIT 20;

```

WHAT WE GET

RANK leaves gaps after ties (1,1,3), DENSE_RANK does not (1,1,2). Shows the worst-offending merchants for a block-list.

Q31

Question: (NTILE) Split customers into 10 spend deciles for risk tiering.

```

SELECT
    cc_num,
    total_spend,
    NTILE(10) OVER (ORDER BY total_spend DESC) AS spend_decile
FROM (
    SELECT cc_num, SUM(amt) AS total_spend
    FROM Transactions
    GROUP BY cc_num
) s
ORDER BY spend_decile, total_spend DESC
LIMIT 30;

```

WHAT WE GET

NTILE(10) divides customers into ten equal groups. Decile 1 = the biggest spenders (highest financial exposure) - a ready segmentation for tiered monitoring.

Q32

Question: (LAG) For each customer, compare every transaction amount to their PREVIOUS transaction - detect sudden jumps.

```

SELECT
    cc_num,
    trans_date_trans_time,
    amt,
    LAG(amt) OVER (PARTITION BY cc_num ORDER BY trans_date_trans_time) AS prev_amt,
    amt - LAG(amt) OVER (PARTITION BY cc_num ORDER BY trans_date_trans_time) AS amt_change
FROM Transactions
WHERE cc_num = (SELECT cc_num FROM Transactions LIMIT 1) -- one customer for readability
ORDER BY trans_date_trans_time
LIMIT 30;

```

WHAT WE GET

LAG pulls the prior row's value into the current row. A large positive amt_change (a spend spike right after small buys) is a textbook fraud pattern.

Q33

Question: (LEAD) Measure the time gap (in minutes) to each customer's NEXT transaction - a velocity signal.

```

SELECT
  cc_num,
  trans_date_trans_time,
  LEAD(trans_date_trans_time) OVER (
    PARTITION BY cc_num ORDER BY trans_date_trans_time
  ) AS next_txn_time,
  TIMESTAMPDIFF(
    MINUTE,
    trans_date_trans_time,
    LEAD(trans_date_trans_time) OVER (
      PARTITION BY cc_num ORDER BY trans_date_trans_time
    ) AS mins_to_next
  ) AS mins_to_next
FROM Transactions
WHERE cc_num = (SELECT cc_num FROM Transactions LIMIT 1)
ORDER BY trans_date_trans_time
LIMIT 30;

```

WHAT WE GET

LEAD looks forward to the next row. Very small mins_to_next (several purchases within minutes) indicates card-testing / rapid-fire fraud.

Q34

Question: (FIRST_VALUE / LAST_VALUE) Show each customer's first-ever and most-recent transaction amount side by side.

```

SELECT DISTINCT
  cc_num,
  FIRST_VALUE(amt) OVER w AS first_txn_amt,
  LAST_VALUE(amt) OVER w AS latest_txn_amt
FROM Transactions
WINDOW w AS (
  PARTITION BY cc_num
  ORDER BY trans_date_trans_time
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
LIMIT 20;

```

WHAT WE GET

LAST_VALUE needs the full-frame clause (UNBOUNDED ... FOLLOWING) or it only "sees" up to the current row. Compares onboarding behaviour vs current behaviour per customer.

Q35

Question: (Running total) Cumulative fraud LOSS over time (daily).

```

SELECT
  txn_date,
  daily_fraud_amt,
  SUM(daily_fraud_amt) OVER (ORDER BY txn_date) AS cumulative_fraud_amt
FROM (
  SELECT
    DATE(trans_date_trans_time) AS txn_date,
    SUM(CASE WHEN is_fraud=1 THEN amt ELSE 0 END) AS daily_fraud_amt
  FROM Transactions
  GROUP BY DATE(trans_date_trans_time)
) d
ORDER BY txn_date
LIMIT 40;

```

WHAT WE GET

SUM() OVER (ORDER BY date) accumulates day by day - the running total of money lost to fraud, exactly the curve a finance team tracks.

Q36

Question: (Moving average) 7-day moving average of daily fraud count to smooth out day-to-day noise.

```
SELECT
    txn_date,
    daily_fraud_cnt,
    ROUND(AVG(daily_fraud_cnt) OVER (
        ORDER BY txn_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ), 2) AS moving_avg_7d
FROM (
    SELECT
        DATE(trans_date_trans_time) AS txn_date,
        SUM(is_fraud)                AS daily_fraud_cnt
    FROM Transactions
    GROUP BY DATE(trans_date_trans_time)
) d
ORDER BY txn_date
LIMIT 40;
```

WHAT WE GET

A framed AVG() over the trailing 7 rows smooths spiky daily counts into a trend line - standard for detecting a genuine upward drift vs random noise.

Q37

Question: (Share of total) Each category's percentage share of ALL fraud.

```
SELECT
    m.category,
    SUM(t.is_fraud) AS fraud_txns,
    ROUND(100 * SUM(t.is_fraud)
        / SUM(SUM(t.is_fraud)) OVER (), 2) AS pct_of_all_fraud
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
GROUP BY m.category
ORDER BY fraud_txns DESC;
```

WHAT WE GET

SUM(SUM(...)) OVER () gives the grand total as a window over the grouped rows, so each category's fraud is expressed as a % of the whole - a Pareto view of where fraud volume lives.

CTEs & Advanced Analytics

Q38

Question: (Simple CTE) Summarise fraud per category, then keep only the categories whose fraud rate beats the overall rate - written readably with a CTE instead of a nested subquery.

```

WITH category_stats AS (
    SELECT
        m.category,
        COUNT(*) AS total_txns,
        SUM(t.is_fraud) AS fraud_txns,
        100 * SUM(t.is_fraud) / COUNT(*) AS fraud_rate
    FROM Transactions t
    JOIN Merchants m ON t.merchant_id = m.merchant_id
    GROUP BY m.category
)
SELECT category, total_txns, fraud_txns, ROUND(fraud_rate,3) AS fraud_rate_pct
FROM category_stats
WHERE fraud_rate > (SELECT AVG(fraud_rate) FROM category_stats)
ORDER BY fraud_rate DESC;

```

WHAT WE GET

A CTE names an intermediate result so the final query reads top-to-bottom. Same output as a subquery, far more maintainable.

Q39

Question: (Chained CTEs) Build a customer risk score in stages: spend profile -> fraud profile -> combined score.

```

WITH spend AS (
    SELECT cc_num, COUNT(*) AS txns, SUM(amt) AS total_spend
    FROM Transactions
    GROUP BY cc_num
),
fraud AS (
    SELECT cc_num,
        SUM(is_fraud) AS fraud_cnt,
        SUM(CASE WHEN is_fraud=1 THEN amt ELSE 0 END) AS fraud_amt
    FROM Transactions
    GROUP BY cc_num
),
scored AS (
    SELECT
        s.cc_num,
        s.txns,
        s.total_spend,
        f.fraud_cnt,
        f.fraud_amt,
        ROUND(100 * f.fraud_cnt / s.txns, 2) AS pct_txns_fraud
    FROM spend s
    JOIN fraud f ON s.cc_num = f.cc_num
)
SELECT *
FROM scored
WHERE fraud_cnt > 0
ORDER BY pct_txns_fraud DESC, fraud_amt DESC
LIMIT 20;

```

WHAT WE GET

Multiple CTEs chain like a pipeline - each stage builds on the last. Produces a ranked list of the customers most compromised by fraud, with the logic broken into clear steps.

Q40

Question: (Recursive CTE) Generate a continuous calendar of dates so the daily fraud trend has NO missing days (zero-fill the gaps).

```

WITH RECURSIVE date_series AS (
  SELECT (SELECT MIN(DATE(trans_date_trans_time)) FROM Transactions) AS d
  UNION ALL
  SELECT d + INTERVAL 1 DAY
  FROM date_series
  WHERE d < (SELECT MAX(DATE(trans_date_trans_time)) FROM Transactions)
)
SELECT
  ds.d
  ds.d AS calendar_date,
  IFNULL(f.fraud_cnt, 0) AS fraud_cnt
FROM date_series ds
LEFT JOIN (
  SELECT DATE(trans_date_trans_time) AS d, SUM(is_fraud) AS fraud_cnt
  FROM Transactions
  GROUP BY DATE(trans_date_trans_time)
) f ON ds.d = f.d
ORDER BY ds.d
LIMIT 60;

```

WHAT WE GET

A recursive CTE builds a row per calendar day between the first and last transaction. LEFT JOINing the real counts and IFNULL-ing to 0 gives a gap-free series - essential before charting a trend.

Note: If the range is large, raise the limit first: SET SESSION cte_max_recursion_depth = 10000;

Q41

Question: (Recursive CTE - pure demo) Generate numbers 1..20.

```

WITH RECURSIVE nums AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 20
)
SELECT n FROM nums;

```

WHAT WE GET

The minimal recursive pattern - an anchor row (1) plus a recursive step (n+1) with a stop condition (n < 20). The mental model behind Q40's date generator.

Advanced Analytics

Q42

Question: (Haversine) Does physical distance between the cardholder's home and the merchant correlate with fraud?

```

WITH dist AS (
  SELECT
    t.is_fraud,
    6371 * ACOS(
      LEAST(1.0,
        COS(RADIANS(c.lat)) * COS(RADIANS(t.merch_lat)) *
        COS(RADIANS(t.merch_long) - RADIANS(c.`long`)) +
        SIN(RADIANS(c.lat)) * SIN(RADIANS(t.merch_lat))
      )
    ) AS distance_km
  FROM Transactions t
  JOIN Customers c ON t.cc_num = c.cc_num
)
SELECT
  CASE WHEN is_fraud=1 THEN 'Fraud' ELSE 'Legitimate' END AS txn_type,
  COUNT(*) AS txns,
  ROUND(AVG(distance_km), 2) AS avg_distance_km,
  ROUND(MAX(distance_km), 2) AS max_distance_km
FROM dist
GROUP BY is_fraud;

```

WHAT WE GET

The haversine formula converts two lat/long pairs into great-circle km. Comparing the average distance for fraud vs legit transactions tests whether "far from home" is a red flag.

Q43

Question: (Z-score anomaly) Flag transactions that sit far above a customer's own typical spend (statistical outliers).

```

WITH cust_stats AS (
  SELECT cc_num, AVG(amt) AS mean_amt, STDDEV_SAMP(amt) AS sd_amt
  FROM Transactions
  GROUP BY cc_num
)
SELECT
  t.trans_num,
  t.cc_num,
  t.amt,
  ROUND(cs.mean_amt, 2) AS cust_mean,
  ROUND((t.amt - cs.mean_amt) / NULLIF(cs.sd_amt, 0), 2) AS z_score,
  t.is_fraud
FROM Transactions t
JOIN cust_stats cs ON t.cc_num = cs.cc_num
WHERE cs.sd_amt > 0
  AND (t.amt - cs.mean_amt) / cs.sd_amt > 3 -- more than 3 SD above their mean
ORDER BY z_score DESC
LIMIT 25;

```

WHAT WE GET

Z-score = (value - mean) / std-dev, computed PER customer. A z-score above 3 means the amount is extreme relative to that person's history - and these rows skew heavily toward is_fraud=1.

Q44

Question: (Cohort analysis) Group customers by the MONTH of their first transaction, and track how much fraud each cohort suffers.

```

WITH first_seen AS (
  SELECT cc_num,
         DATE_FORMAT(MIN(trans_date_trans_time), '%Y-%m') AS cohort_month
  FROM Transactions
  GROUP BY cc_num
)
SELECT
  fs.cohort_month,
  COUNT(DISTINCT fs.cc_num) AS cohort_size,
  SUM(t.is_fraud) AS fraud_txns,
  ROUND(100 * SUM(t.is_fraud)/COUNT(*), 3) AS fraud_rate_pct
FROM first_seen fs
JOIN Transactions t ON fs.cc_num = t.cc_num
GROUP BY fs.cohort_month
ORDER BY fs.cohort_month;

```

WHAT WE GET

Cohorting by acquisition month shows whether customers who joined in certain months are structurally more fraud-exposed - a retention/onboarding-quality lens on fraud.

Q45

Question: (Repeat victims) Which customers were defrauded more than once?

```

SELECT
  c.cc_num,
  CONCAT(c.first, ' ', c.last) AS customer_name,
  SUM(t.is_fraud) AS fraud_incidents,
  ROUND(SUM(CASE WHEN t.is_fraud=1 THEN t.amt ELSE 0 END),2) AS total_fraud_loss
FROM Transactions t
JOIN Customers c ON t.cc_num = c.cc_num
GROUP BY c.cc_num, customer_name
HAVING fraud_incidents > 1
ORDER BY fraud_incidents DESC, total_fraud_loss DESC
LIMIT 20;

```

WHAT WE GET

Repeat victims are high-priority for card reissue / account review. HAVING filters to customers hit 2+ times and ranks them by both frequency and money lost.

Q46

Question: (Velocity check) Customers with 3+ transactions inside any single hour - a rapid-fire pattern typical of card testing.

```

SELECT
  cc_num,
  DATE_FORMAT(trans_date_trans_time, '%Y-%m-%d %H') AS txn_hour_bucket,
  COUNT(*) AS txns_in_hour,
  SUM(is_fraud) AS frauds_in_hour
FROM Transactions
GROUP BY cc_num, DATE_FORMAT(trans_date_trans_time, '%Y-%m-%d %H')
HAVING txns_in_hour >= 3
ORDER BY txns_in_hour DESC
LIMIT 25;

```

WHAT WE GET

Bucketing by customer + hour and keeping groups of 3+ surfaces bursts of activity. High-velocity clusters are a strong, real-time-friendly fraud indicator.

Views, Stored Procedures & Transactions

Views

Q47

Question: (VIEW) A reusable daily fraud summary so dashboards/queries never repeat the aggregation logic.

```
CREATE OR REPLACE VIEW v_daily_fraud_summary AS
SELECT
    DATE(trans_date_trans_time)          AS txn_date,
    COUNT(*)                             AS total_txns,
    SUM(is_fraud)                         AS fraud_txns,
    ROUND(100 * SUM(is_fraud)/COUNT(*), 3) AS fraud_rate_pct,
    ROUND(SUM(CASE WHEN is_fraud=1 THEN amt ELSE 0 END),2) AS fraud_loss
FROM Transactions
GROUP BY DATE(trans_date_trans_time);

-- use it like a table:
SELECT * FROM v_daily_fraud_summary ORDER BY fraud_loss DESC LIMIT 10;
```

WHAT WE GET

A VIEW stores the query, not the data. Anyone can now "SELECT * FROM v_daily_fraud_summary" without rewriting the aggregation - one source of truth for daily fraud numbers.

Q48

Question: (VIEW) A high-risk customer view - customers with any fraud, ready to feed a review queue.

```
CREATE OR REPLACE VIEW v_high_risk_customers AS
SELECT
    c.cc_num,
    CONCAT(c.first, ' ', c.last)          AS customer_name,
    c.state,
    COUNT(*)                             AS total_txns,
    SUM(t.is_fraud)                       AS fraud_txns,
    ROUND(SUM(CASE WHEN t.is_fraud=1 THEN t.amt ELSE 0 END),2) AS fraud_loss
FROM Customers c
JOIN Transactions t ON c.cc_num = t.cc_num
GROUP BY c.cc_num, customer_name, c.state
HAVING fraud_txns > 0;

SELECT * FROM v_high_risk_customers ORDER BY fraud_loss DESC LIMIT 10;
```

WHAT WE GET

Encapsulates the "who to investigate" logic. The review team queries a clean, named object instead of a 10-line aggregate.

Stored Procedures

Q49

Question: (Stored Procedure) Parameterised fraud report for any date range.

```
DROP PROCEDURE IF EXISTS sp_fraud_report_by_daterange;
DELIMITER $$
CREATE PROCEDURE sp_fraud_report_by_daterange(
    IN p_start_date DATE,
    IN p_end_date DATE
)
BEGIN
    SELECT
        m.category,
        COUNT(*) AS total_txns,
        SUM(t.is_fraud) AS fraud_txns,
        ROUND(100 * SUM(t.is_fraud)/COUNT(*), 3) AS fraud_rate_pct,
        ROUND(SUM(CASE WHEN t.is_fraud=1 THEN t.amt ELSE 0 END),2) AS fraud_loss
    FROM Transactions t
    JOIN Merchants m ON t.merchant_id = m.merchant_id
    WHERE t.trans_date_trans_time >= p_start_date
        AND t.trans_date_trans_time < p_end_date + INTERVAL 1 DAY
    GROUP BY m.category
    ORDER BY fraud_loss DESC;
END $$
DELIMITER ;

-- call it:
CALL sp_fraud_report_by_daterange('2019-01-01', '2019-03-31');
```

WHAT WE GET

A stored procedure packages parameterised logic on the server. Analysts just CALL it with two dates - no need to hand-edit the query, and the logic stays consistent for everyone.

Q50

Question: (Stored Procedure with OUT param) Return the total fraud loss for a single category into an output variable.

```
DROP PROCEDURE IF EXISTS sp_category_fraud_loss;
DELIMITER $$
CREATE PROCEDURE sp_category_fraud_loss(
    IN p_category VARCHAR(60),
    OUT p_total_loss DECIMAL(14,2)
)
BEGIN
    SELECT SUM(CASE WHEN t.is_fraud=1 THEN t.amt ELSE 0 END)
        INTO p_total_loss
    FROM Transactions t
    JOIN Merchants m ON t.merchant_id = m.merchant_id
    WHERE m.category = p_category;
END $$
DELIMITER ;

-- call it and read the OUT value:
CALL sp_category_fraud_loss('shopping_net', @loss);
SELECT @loss AS shopping_net_fraud_loss;
```

WHAT WE GET

OUT parameters let a procedure hand a computed scalar back to the caller - handy when the result feeds another step rather than being displayed directly.

Transactions (Acid: Commit / Rollback)

Q51

Question: (ROLLBACK) A transaction was wrongly flagged as fraud. Start a transaction, correct it, inspect, then ROLL BACK to undo the change - proving changes are not permanent until committed.

```
START TRANSACTION;

UPDATE Transactions
SET is_fraud = 0
WHERE trans_num = (SELECT trans_num FROM (
    SELECT trans_num FROM Transactions WHERE is_fraud = 1 LIMIT 1
) x);

-- check the change is visible INSIDE the transaction:
SELECT trans_num, is_fraud
FROM Transactions
WHERE is_fraud = 0
ORDER BY trans_num
LIMIT 1;

ROLLBACK; -- undo everything since START TRANSACTION
```

WHAT WE GET

ROLLBACK discards uncommitted work - the row reverts to is_fraud=1. This demonstrates Atomicity: nothing is saved until COMMIT, so a mistake mid-operation costs nothing.

Q52

Question: (COMMIT) The correct way to make a genuine, permanent correction.

```
START TRANSACTION;

-- (Example: mark a confirmed legitimate transaction. Adjust the WHERE to a
-- real trans_num you have verified before running for real.)
UPDATE Transactions
SET is_fraud = 0
WHERE trans_num = 'PUT_A_VERIFIED_TRANS_NUM_HERE';

COMMIT; -- make it permanent
```

WHAT WE GET

COMMIT persists the change so it survives disconnects/restarts (Durability). START TRANSACTION + COMMIT is the safe pattern for any manual data correction - you get a chance to verify, and can ROLLBACK instead if something looks wrong.

Performance & Indexing

Q53

Question: (EXPLAIN - before) Look at the plan for a common lookup: all transactions for one customer. Without an index this is a full table scan.

```
EXPLAIN
SELECT * FROM Transactions
WHERE cc_num = 4396091624673316;
```

WHAT WE GET

Read the "type" and "rows" columns. type = ALL means a full scan; rows ~ 1.3M means MySQL expects to examine every row. That is the cost we are about to remove.

Q54

Question: (CREATE INDEX) Add the indexes that support our most frequent access patterns: by customer, by merchant, by date.

```
CREATE INDEX idx_txn_cc_num      ON Transactions (cc_num);
CREATE INDEX idx_txn_merchant    ON Transactions (merchant_id);
CREATE INDEX idx_txn_date        ON Transactions (trans_date_trans_time);
CREATE INDEX idx_txn_is_fraud    ON Transactions (is_fraud);
```

WHAT WE GET

An index is a sorted lookup structure (B-tree). These four cover the columns we filter and join on most. Note the trade-off: indexes speed up reads but slightly slow down inserts/updates and use extra disk.

Q55

Question: (EXPLAIN - after) Re-check the same customer lookup. It should now use the index instead of scanning the whole table.

```
EXPLAIN
SELECT * FROM Transactions
WHERE cc_num = 4396091624673316;
```

WHAT WE GET

type should change from ALL to ref, key should show idx_txn_cc_num, and rows should drop from ~1.3M to a few hundred. Same query, a fraction of the work - the whole point of indexing.

Q56

Question: (Composite index) A date-range + fraud filter is common. A composite index on (is_fraud, trans_date_trans_time) serves it in one structure.

```
CREATE INDEX idx_fraud_date ON Transactions (is_fraud, trans_date_trans_time);

EXPLAIN
SELECT COUNT(*), SUM(amt)
FROM Transactions
WHERE is_fraud = 1
      AND trans_date_trans_time BETWEEN '2019-06-01' AND '2019-06-30';
```

WHAT WE GET

Column order in a composite index matters - the most selective / equality column (is_fraud) comes first, then the range column (date). MySQL can satisfy both predicates from this one index.

Q57

Question: (Inspect indexes) List every index now on the Transactions table.

```
SHOW INDEX FROM Transactions;
```

WHAT WE GET

Confirms which indexes exist (name, columns, cardinality). High cardinality = many distinct values = a more useful index. Good final check that the tuning above was applied.

